

Ain Shams University - Faculty of Engineering
CSE336 Software Design Patterns - Fall 2026

Software Requirements Specification

Impact Hub

Non-Profit Organization Management System

Major Task - Phase 1 Submission

Team Members

Name	Student ID
Saifeldin Tamer ElSayes	22P0233
Mohamed Emad Eldeen	22P0065
Ahmed Khaled Hamdy	22P0024
Yousef Sameh Fathi	22P0010
Seif Eldeen Ibrahim	22P0023
Mohamed Khaled Mohamed Anwar	22P0041

Version 1.0 - November 2026

Contents

1. Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Definitions and Abbreviations 3
1.4 References 3
1.5 Document Overview 4
2. Overall Description 4
2.1 Product Perspective 4
2.2 Product Functions 4
2.3 User Classes 4
2.4 Operating Environment 5
2.5 Design and Implementation Constraints 5
2.6 Assumptions and Dependencies 5
3. Functional Requirements 6
3.1 People and Accounts 6
3.2 Donation Management 6
3.3 Event Management 7
3.4 Volunteer Management 7
3.5 Beneficiary Management 8
3.6 Communication 8
3.7 Administration - Lookup Tables 8
3.8 Dashboard 8
4. Client Requirements 9
5. Non-Functional Requirements 9
6. System Design 11
6.1 Architecture - Model View Controller 11
6.2 Class Diagrams 12
6.3 Design Patterns Applied 20
6.4 CRUD Lookup Tables 23
6.5 Main Design Decisions 23
7. Work Distribution 24
8. Traceability to the Phase 1 Grading Criteria 24
9. Running the System 25
9.1 Build and Run 25
9.2 Sample Data 25
9.3 Demonstration Scenario 26

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes Impact Hub, a management system for a non-profit organization. It defines what the system must do, the quality attributes it must have, and how its object-oriented design uses design patterns to stay flexible, maintainable and reusable.

The document is written for the course instructor and teaching assistants who evaluate the project, and for the six team members who build it. It covers Phase 1 of the CSE336 major task.

The class diagrams and the Java code described here are the same design: every class in the diagrams exists in the code with the same attributes, operations and relationships.

1.2 Scope

Impact Hub supports the daily work of a non-profit: managing people and accounts, receiving donations, organizing events and volunteers, supporting beneficiaries and communicating with all of them. The project is delivered in two phases:

- **Phase 1** (this document): the complete class design and a working Java implementation of all five modules. It uses the patterns taught so far - Strategy, Observer, Decorator, Factory and Singleton - organized with the MVC architecture, and has editable CRUD lookup tables. The user interface is a deliberately simple set of JavaFX windows whose only purpose is to show that every feature works.
- **Phase 2**: the remaining patterns (Command, Adapter, Facade, Template Method, Iterator, State, Proxy), admin pages, and integration with real payment, e-mail and SMS providers.

Requirements that belong to Phase 2 are still listed in Section 3 so the design stays complete, and each one is marked with its phase.

1.3 Definitions and Abbreviations

Term	Meaning
NPO	Non-profit organization
Donor	A person or company that gives money (online or by check) or goods (in-kind donation)
Volunteer	A person who gives their time at the organization's events
Beneficiary	A person, with their household, whom the organization supports
Assignment	A volunteer working at one event in a given role
Distribution	A quantity of a resource given to a beneficiary
Lookup table	A small reference table (skills, resource categories) that staff edit at runtime
CRUD	Create, Read, Update, Delete
DAO	Data Access Object - the class that stores and retrieves records
MVC	Model-View-Controller architecture
EGP	Egyptian Pound, the currency used for all amounts

1.4 References

- CSE336 Major Assessment Task, Part 1 - project brief and evaluation criteria.
- E. Freeman, E. Robson, B. Bates, K. Sierra, *Head First Design Patterns*, O'Reilly.

- E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, 1995.
- CSE336 lecture slides (Strategy, Observer, Decorator, Factory, Singleton) and labs.

1.5 Document Overview

Section 2 describes the product and its users. Section 3 lists the functional requirements per module and Section 4 the client requirements that extend the basic brief. Section 5 gives the non-functional requirements. Section 6 presents the design: architecture, class diagrams, each design pattern, the lookup tables and the main design decisions. Section 7 shows how the work is divided in the team, Section 8 maps every Phase 1 grading item to the diagram and code that satisfies it, and Section 9 explains how to run the system.

2. Overall Description

2.1 Product Perspective

Impact Hub is a new, self-contained desktop system. In Phase 1 it is a Java application with a basic JavaFX interface: the application opens on a dashboard, and after logging in or signing up the user opens one window per module from it. Its data is kept in a local SQLite database file (`impacthub.db`) that is filled with sample data on the first run, so every feature can be demonstrated immediately. Payments and message delivery are simulated: no real card gateway, bank, e-mail server or SMS provider is contacted, and every delivered message is printed in the terminal with its channel.

2.2 Product Functions

Module	Main functions
People and accounts	Sign up as donor, volunteer or beneficiary; log in and out with a hashed password; search and delete people; each person has a preferred contact channel
Donation management	Record online, check and in-kind donations; list donations; issue receipts with optional tax information and thank-you note; thank donors automatically; produce yearly tax statements
Event management	Create fundraisers, workshops and outreach programs; register attendees; sell tickets with extras; reschedule events and notify attendees; track attendance and send reminders
Volunteer management	Store skills, availability and preferences; assign volunteers to events with a role; track hours; issue certificates
Beneficiary management	Store households and their needs, distribute resources from stock, evaluate the impact of programs
Communication	Notify people automatically through their preferred channel (e-mail, SMS or social media)
Administration	Maintain the lookup tables: volunteer skills and resource categories
Dashboard	Open every service, log in, sign up and log out from one window; show the last 10 actions from all services

2.3 User Classes

User	Description
Administrator	Maintains the lookup tables
Staff member	Records donations and distributions, manages events and volunteer assignments
Donor	Gives money or goods, buys event tickets, receives receipts and thank-you messages
Volunteer	Gives time at events, receives event updates and certificates

User	Description
Beneficiary	Receives support and updates about events that concern them

In Phase 1 every logged-in user can use every service; restricting services by user class is Phase 2 (FR-P10).

2.4 Operating Environment

Java 21 (JDK) built with Apache Maven, on Windows, Linux or macOS. Maven downloads JavaFX 21 and the SQLite JDBC driver, so nothing else has to be installed. The project opens directly in IntelliJ IDEA or NetBeans as a Maven project.

2.5 Design and Implementation Constraints

- The design must apply the design patterns taught in the course and follow object-oriented principles: encapsulation, program to an interface, favour composition over inheritance, and the open/closed principle.
- The code contains exactly the classes of the class diagram; the only extra class is `App`, the JavaFX entry point that wires the objects together and opens the dashboard.
- Views only talk to their controller; controllers call the services, and use the generic DAO directly to find a record by id; only views display results.
- Input rules are checked in the services through the shared `Validation` class, so no window can skip them.
- All data goes through one access point (the `DatabaseManager` Singleton) and one generic DAO.

2.6 Assumptions and Dependencies

- All amounts are in Egyptian Pounds (EGP).
- Online and check payments are simulated in Phase 1; integration with a payment provider is Phase 2.
- E-mail, SMS and social-media delivery are simulated by printing the message in the terminal with its channel and address; real providers are Phase 2.

3. Functional Requirements

Each requirement has an identifier, a description and the phase in which it is delivered. "Phase 1 - service only" means the operation is implemented and working in the model, but the Phase 1 user interface has no button for it.

3.1 People and Accounts

ID	Requirement	Status
FR-P1	The system shall let a person sign up as a donor, volunteer or beneficiary with name, e-mail, phone and password.	Phase 1 - implemented
FR-P2	The system shall refuse a sign-up without a name, e-mail or password.	Phase 1 - implemented
FR-P3	The system shall store only a SHA-256 hash of each password and log people in by e-mail and password.	Phase 1 - implemented
FR-P4	The system shall let staff search people by name; an empty name lists everyone.	Phase 1 - implemented
FR-P5	The system shall let staff delete a person by id.	Phase 1 - implemented
FR-P6	The system shall keep each person's preferred contact channel (e-mail by default).	Phase 1 - implemented
FR-P7	The system shall let a person edit their contact details and preferred channel, and enter their tax id, skills, availability, address, family size and needs, from the interface.	Phase 2
FR-P8	The system shall open no service until the user has logged in or signed up; signing up logs the new person in, and logging out closes every open service window.	Phase 1 - implemented
FR-P9	The system shall refuse a sign-up with an invalid e-mail, a phone that is not 7 to 15 digits, a password shorter than 4 characters or an e-mail that is already registered, and shall refuse deleting your own account while logged in.	Phase 1 - implemented
FR-P10	The system shall restrict each service by user class (administrator, staff, donor, volunteer, beneficiary).	Phase 2

3.2 Donation Management

ID	Requirement	Status
FR-D1	The system shall store donor details: donor type (individual or corporate), tax id and total donated.	Phase 1 - service only
FR-D2	The system shall record a donation of type online, check or in-kind for a donor with an amount.	Phase 1 - implemented
FR-D3	The system shall reject a donation whose amount is not more than 0, is not a number or is above 1,000,000,000.	Phase 1 - implemented
FR-D4	The system shall keep each type's own details: transaction id and gateway (online), check number, bank and clearing date (check), item description, quantity and estimated value (in-kind).	Phase 1 - implemented
FR-D5	The system shall list all recorded donations, including those of donors who were deleted later.	Phase 1 - implemented
FR-D6	The system shall issue a receipt for any recorded donation, with optional tax information and/or thank-you note, and print a receipt automatically when a donation is recorded.	Phase 1 - implemented

ID	Requirement	Status
FR-D7	The system shall keep each donor's total donated up to date and saved, and the organization's total raised up to date while the application runs.	Phase 1 - implemented
FR-D8	The system shall produce a yearly donation statement for a donor's tax records.	Phase 1 - service only
FR-D9	The system shall connect to a real payment gateway and bank clearing service.	Phase 2
FR-D10	The system shall show the organization's total raised and keep it when the application is closed.	Phase 2

3.3 Event Management

ID	Requirement	Status
FR-E1	The system shall create fundraisers (target amount), workshops (capacity, trainer) and outreach programs (location, team size) with a required name and venue and a date in the future.	Phase 1 - implemented
FR-E2	The system shall register people for an event and never register the same person twice.	Phase 1 - implemented
FR-E3	The system shall sell tickets (100 EGP) with optional VIP seating (+150 EGP) and dinner (+40 EGP); the holder is registered for the event, and sold tickets are saved and can be listed per event.	Phase 1 - implemented
FR-E4	The system shall change an event's date to a date in the future and inform every attendee, leaving out people who were deleted.	Phase 1 - implemented
FR-E5	The system shall change an event's venue and send reminders, informing every attendee.	Phase 1 - service only
FR-E6	The system shall mark the attendance of registered people.	Phase 1 - service only
FR-E7	The system shall move events through their status (planned, ongoing, completed, cancelled) and enforce workshop capacity.	Phase 2
FR-E8	The system shall refuse registrations and ticket sales for an event that has already taken place.	Phase 1 - implemented

3.4 Volunteer Management

ID	Requirement	Status
FR-V1	The system shall store volunteers' skills (from the Skill lookup table), availability and preferences, and list them.	Phase 1 - service only
FR-V2	The system shall assign a volunteer to an event with a required role, once per event, and list the event's assignments.	Phase 1 - implemented
FR-V3	The system shall track hours per assignment and per volunteer, refusing hours that are not more than 0, are not a number or are above 24 per entry, and hours for a volunteer who was deleted.	Phase 1 - implemented
FR-V4	The system shall issue a volunteer certificate with the total hours.	Phase 1 - implemented
FR-V5	The system shall find the volunteers who have a given skill.	Phase 1 - service only
FR-V6	The system shall let volunteers book their own shifts from a calendar.	Phase 2

3.5 Beneficiary Management

ID	Requirement	Status
FR-B1	The system shall store beneficiaries with address, family size and a list of needs (category, description, priority), and list them with the resources in stock.	Phase 1 - service only
FR-B2	The system shall record the distribution of a resource to a beneficiary, refusing quantities above the stock.	Phase 1 - implemented
FR-B3	The system shall reduce the stock automatically after each distribution.	Phase 1 - implemented
FR-B4	The system shall evaluate the impact of a program with a method suited to that program (food aid or education); a category stays in its program when it is renamed.	Phase 1 - implemented
FR-B5	The system shall add new resources and restock them from the interface.	Phase 2

3.6 Communication

ID	Requirement	Status
FR-C1	The system shall notify everyone concerned automatically when a donation is recorded or an event changes.	Phase 1 - implemented
FR-C2	The system shall deliver each notification through the recipient's preferred channel - e-mail, SMS or social media - falling back to e-mail when needed.	Phase 1 - implemented
FR-C3	The system shall send real e-mails and SMS messages and post on social-media pages.	Phase 2

3.7 Administration - Lookup Tables

ID	Requirement	Status
FR-L1	The administrator shall create, list, rename and delete volunteer skills, with unique names; a rename reaches the volunteers who have the skill.	Phase 1 - implemented
FR-L2	The administrator shall create, list, rename and delete resource categories, with unique names; a rename reaches the resources and needs in the category.	Phase 1 - implemented
FR-L3	The system shall refuse to delete a lookup row that is still used by a volunteer, resource or need.	Phase 1 - implemented

3.8 Dashboard

ID	Requirement	Status
FR-A1	The system shall open on a dashboard with log in, sign up and log out buttons and one button per service, keeping the service buttons disabled until someone is logged in.	Phase 1 - implemented
FR-A2	The system shall show the last 10 actions from all services on the dashboard, newest first, each with its time, the person who did it and a description, and update the list as actions happen.	Phase 1 - implemented
FR-A3	The system shall keep the actions when the application is closed, and hide them while nobody is logged in.	Phase 1 - implemented
FR-A4	The system shall track only actions that succeeded and changed something, plus logging in and out; refused actions and read-only actions such as searches and lists are not tracked.	Phase 1 - implemented
FR-A5	Closing the dashboard shall close the whole application.	Phase 1 - implemented

4. Client Requirements

The brief lists the basic modules. The requirements below extend it with what the organization's staff, donors, volunteers and beneficiaries need on top of those modules. Each one is implemented in Phase 1 and traced to the classes that satisfy it.

ID	Client requirement	Satisfied by
CR-01	Every person has a preferred way to be contacted - e-mail, SMS or social media - and messages arrive through that channel. New sign-ups use e-mail; choosing it from the interface is FR-P7.	Strategy: NotificationService + Email/Sms/SocialMediaStrategy
CR-02	Every donor is thanked automatically right after donating.	Observer: ThankYouObserver
CR-03	A receipt is printed for every donation without staff asking for it.	Observer: ReceiptObserver
CR-04	A donor decides whether the receipt includes tax information and a thank-you note.	Decorator: TaxInfoDecorator, ThankYouNoteDecorator
CR-05	Each donor's running total is always up to date, and the organization's total raised is kept up to date while the application runs.	Observer: StatsObserver
CR-06	Event tickets can include VIP seating and dinner in any combination.	Decorator: VipDecorator, MealDecorator
CR-07	Attendees hear about date changes without staff contacting each one.	Observer: AttendeeNotificationObserver
CR-08	Nobody can be registered twice for the same event.	Event.register()
CR-09	Distributions can never exceed the stock, and the stock is reduced automatically.	DistributionService, Observer: StockObserver
CR-10	Food aid is measured by the number of people fed; education by how well distributions match recorded needs.	Strategy: FoodAid/EducationImpactStrategy
CR-11	Admins add, rename and remove skills and resource categories without a developer.	CRUD: GenericDAO, LookupController
CR-12	Passwords are stored only as hashes.	PersonService (SHA-256)
CR-13	Data is kept when the application is closed and opened again.	Singleton: DatabaseManager + GenericDAO (SQLite)
CR-14	Staff see the last 10 actions from all services in one place as they happen.	Observer: ActivityLogService, DashboardController, DashboardView
CR-15	Nobody can use a service without logging in or signing up, and logging out closes what was open.	Session, DashboardController
CR-16	Every wrong entry gets a clear message, and bad data never reaches the database.	Validation and the services

5. Non-Functional Requirements

ID	Quality	Requirement
NFR-1	Maintainability	Each class has one responsibility and depends on interfaces or abstract classes where behaviour varies, so a change is made in one place.

ID	Quality	Requirement
NFR-2	Extensibility (open/closed)	A new observer is added by writing one new class and subscribing it. A new delivery channel, impact measure, receipt section or ticket extra is one new class plus a small change where it is chosen (the Channel value, or the option in the service and its window); the contexts and decorators do not change. A new donation or event type is one new class, one line in its factory and one option in its window.
NFR-3	Usability	Each window shows the result of every action in its output area; every invalid entry (non-number, bad date, unknown id) shows a clear message instead of crashing.
NFR-4	Data consistency	All services and controllers share one database connection (Singleton) and the same service objects, so every window sees the same data.
NFR-5	Data integrity	Invalid donations, hours, distributions, sign-ups, events and lookup names are rejected; stock cannot go below zero; a person cannot be registered twice for an event; a lookup row in use cannot be deleted; deleting a person keeps their donations, tickets and distributions as history.
NFR-6	Security	Passwords are stored as SHA-256 hashes; no service opens until someone has logged in or signed up.
NFR-7	Persistence	Data is stored in a local SQLite file and survives restarts; deleting the file restores the sample data.
NFR-8	Portability	Runs on any operating system with Java 21 and Maven; no external services or database server are required.
NFR-9	Performance	Every operation completes instantly for the expected data size (hundreds of records).

6. System Design

6.1 Architecture - Model View Controller

The code is organized in three layers. The user only talks to a View; the Controller receives the user's actions from its view, calls the Model (services and domain classes) and sends the results back to the view.

Layer	Classes and packages (`impacthub.*`)	Responsibility
Model	persistence, people, lookup, notification (+ channels), observer, donation (+ receipt), event (+ ticket), volunteer, beneficiary, dashboard (activity log), Validation - the services, domain classes and the data store	Business data and rules: services, domain classes, patterns and storage.
View	View (interface) and PersonView, DonationView, EventView, VolunteerView, BeneficiaryView, LookupView, DashboardView	The JavaFX windows: read the user's input and display results. Each view creates and talks only to its own controller (1 to 1).
Controller	PersonController, DonationController, EventController, VolunteerController, BeneficiaryController, LookupController, DashboardController	One controller per window: validates the input, calls the services and tells the view what to show.
Entry point	impacthub.App	Creates the services, plugs in the notification strategies, subscribes the observers, adds the sample data and opens the dashboard.

Each view and controller sits in the package of the module it serves (for example `donation.DonationView`), so one team member owns a whole vertical slice; the layer of a class is given by its role, not its package.

Design quality checklist

The class design was checked against the course's list of common class-diagram mistakes:

Rule	How the design meets it
No empty classes	All 78 classes have attributes or operations; enumerations list their values.
No standalone classes	Every class, including each enumeration, takes part in at least one of the 187 relationships.
No inheritance cycles	All hierarchies are trees.
No single inheritance	Every parent has at least two children and every interface at least two implementations, e.g. Person (3), Donation (3), Event (3), Receipt (2), ReceiptDecorator (2), Ticket (2), TicketDecorator (2), View (7), Observer (7), Subject (8), NotificationStrategy (3), ImpactEvaluationStrategy (2).
Use interfaces	Subject, Observer, NotificationStrategy, ImpactEvaluationStrategy, Receipt, Ticket and View are interfaces; clients depend on them rather than on concrete classes.
Balanced relationships	The diagrams use association, aggregation, composition, generalization, realization and dependency.
No system class	There is no all-knowing "System" class; work is split across focused services, controllers and domain classes.
Diagram matches code	Every diagram class exists in the code with the same kind, attributes and operations, and every relationship is present in the code (the "observes" links are the subscriptions that App makes with addObserver()). This was checked automatically against the source.

6.2 Class Diagrams

Seven diagrams are provided: the full system and one focused view per Phase 1 grading item. All of them are generated from one Mermaid model of the code (`docs/class-diagram/NGO_UML_class_diagram.mmd`) and drawn with Graphviz, so they always agree with each other and with the implementation. High-resolution PNG and SVG copies and the source are in the `docs/class-diagram` folder.

Notation: + public, - private; underlined members are static; italic names are abstract classes or interfaces; filled diamond = composition, hollow diamond = aggregation, solid arrow = association, dashed arrow = dependency, hollow triangle = generalization (solid) or realization (dashed).

Figure	Diagram	Phase 1 grading item
Figure 1	Full class diagram - all 78 classes, interfaces and enumerations with their 187 relationships	Class diagram
Figure 2	Notification channel and impact evaluation	Strategy pattern
Figure 3	Notifications from the services and the session, and the activity log	Observer pattern
Figure 4	Receipts and tickets	Decorator pattern
Figure 5	Creating donations and events	Factory pattern
Figure 6	The shared database connection	Singleton pattern
Figure 7	Application layers and lookup tables	MVC and CRUD lookup tables

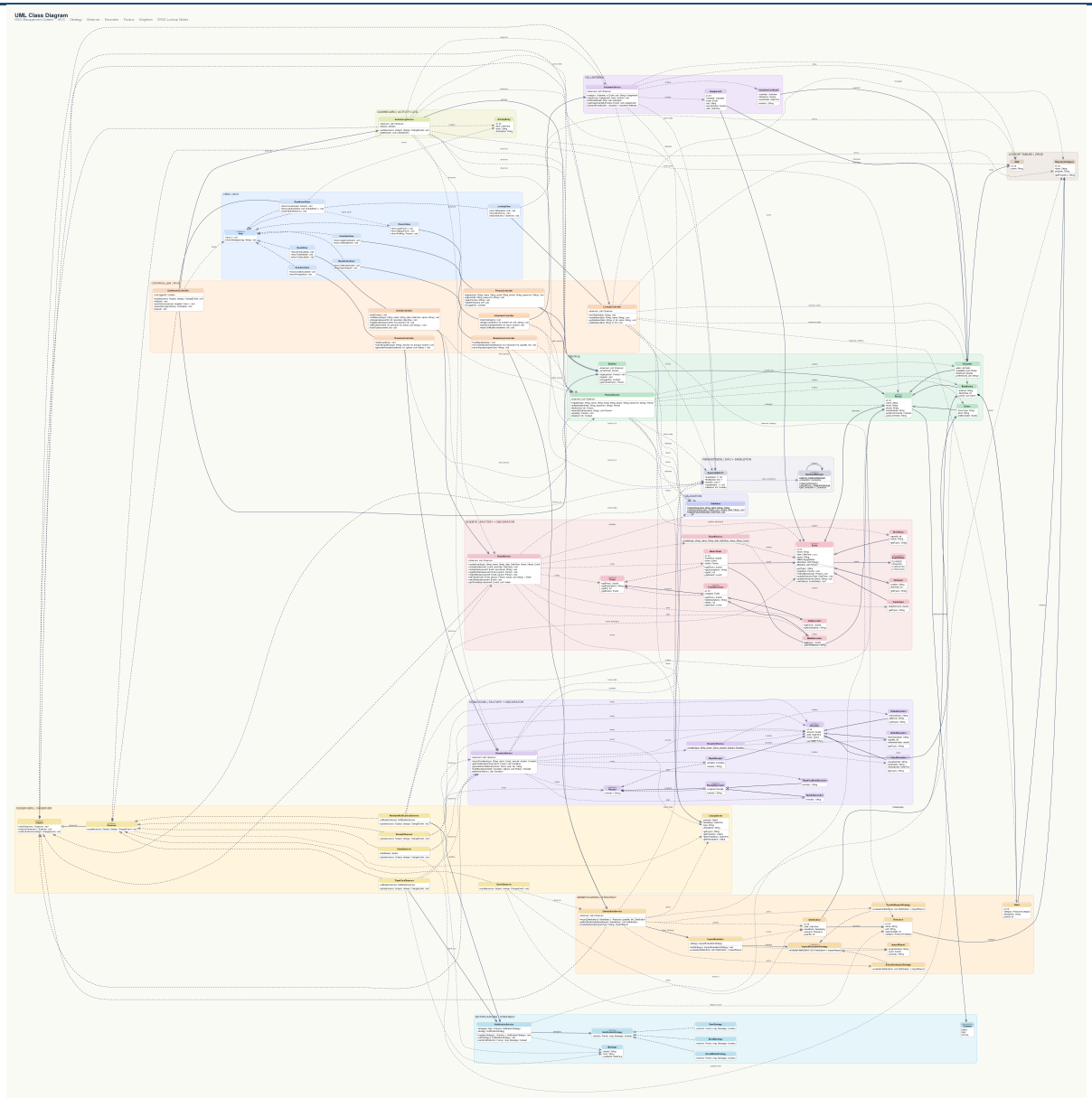


Figure 1: Full class diagram of the system

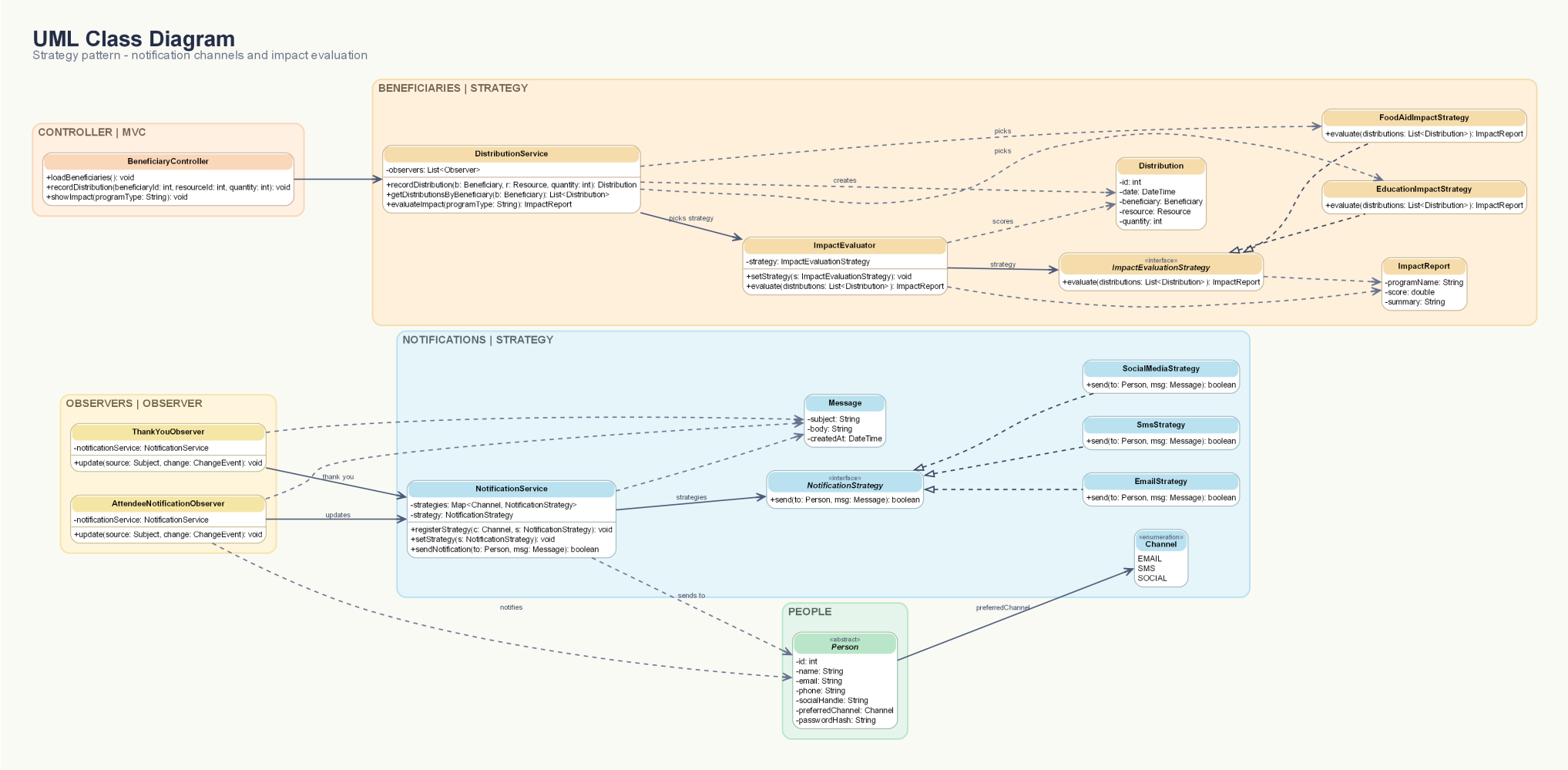


Figure 2: Strategy pattern - notification channels and impact evaluation

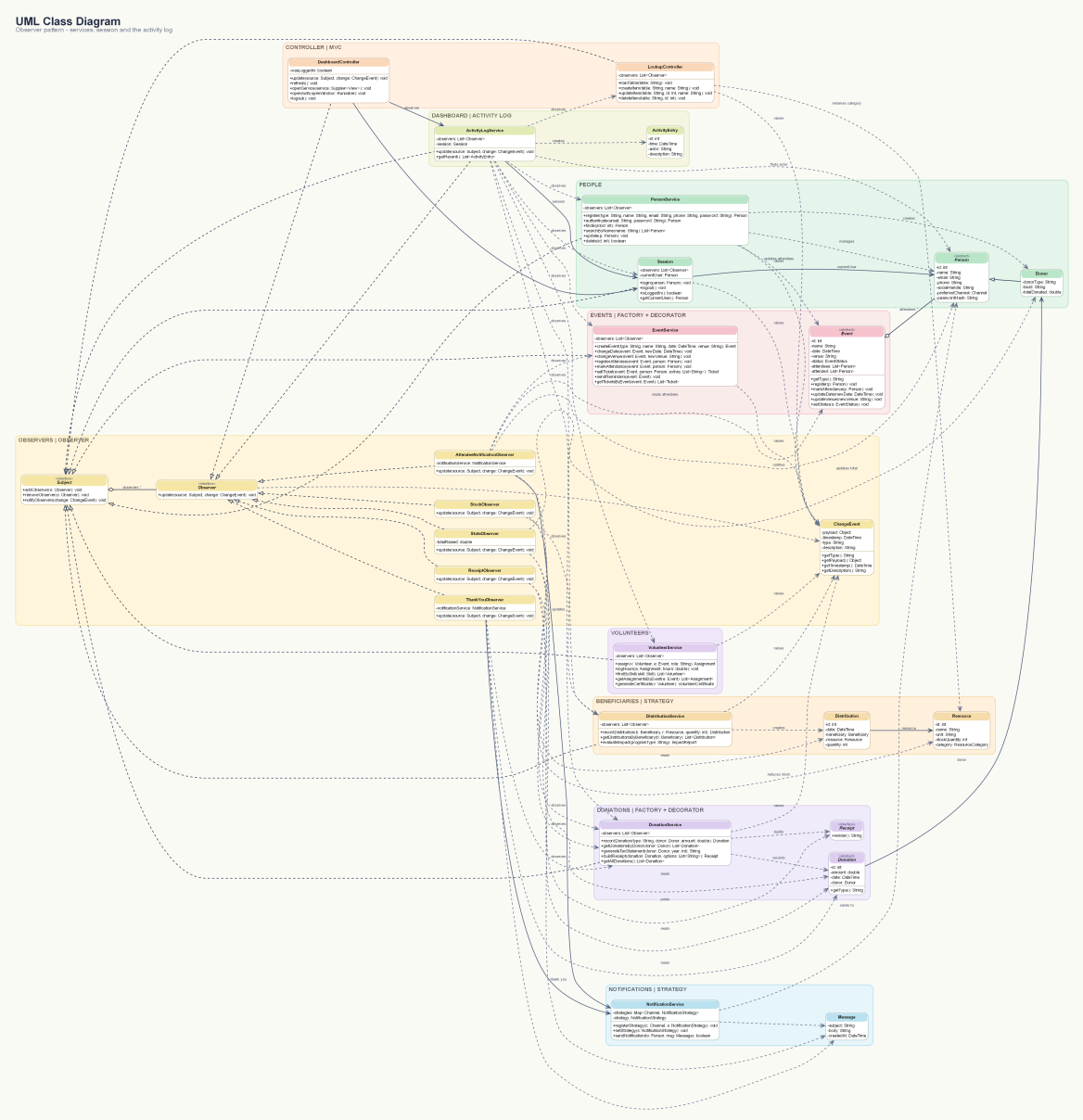


Figure 3: Observer pattern - services, session and the activity log

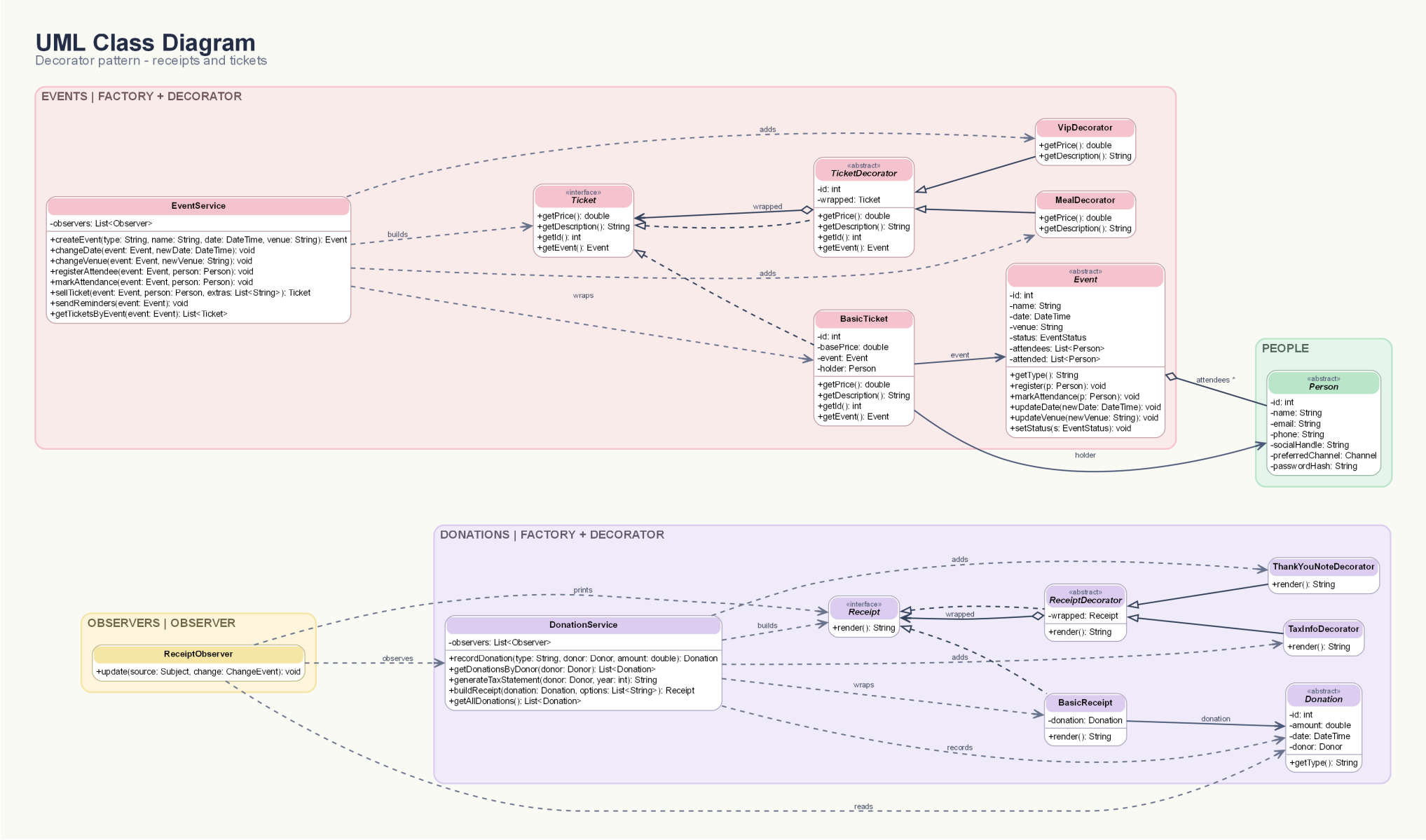


Figure 4: Decorator pattern - receipts and tickets

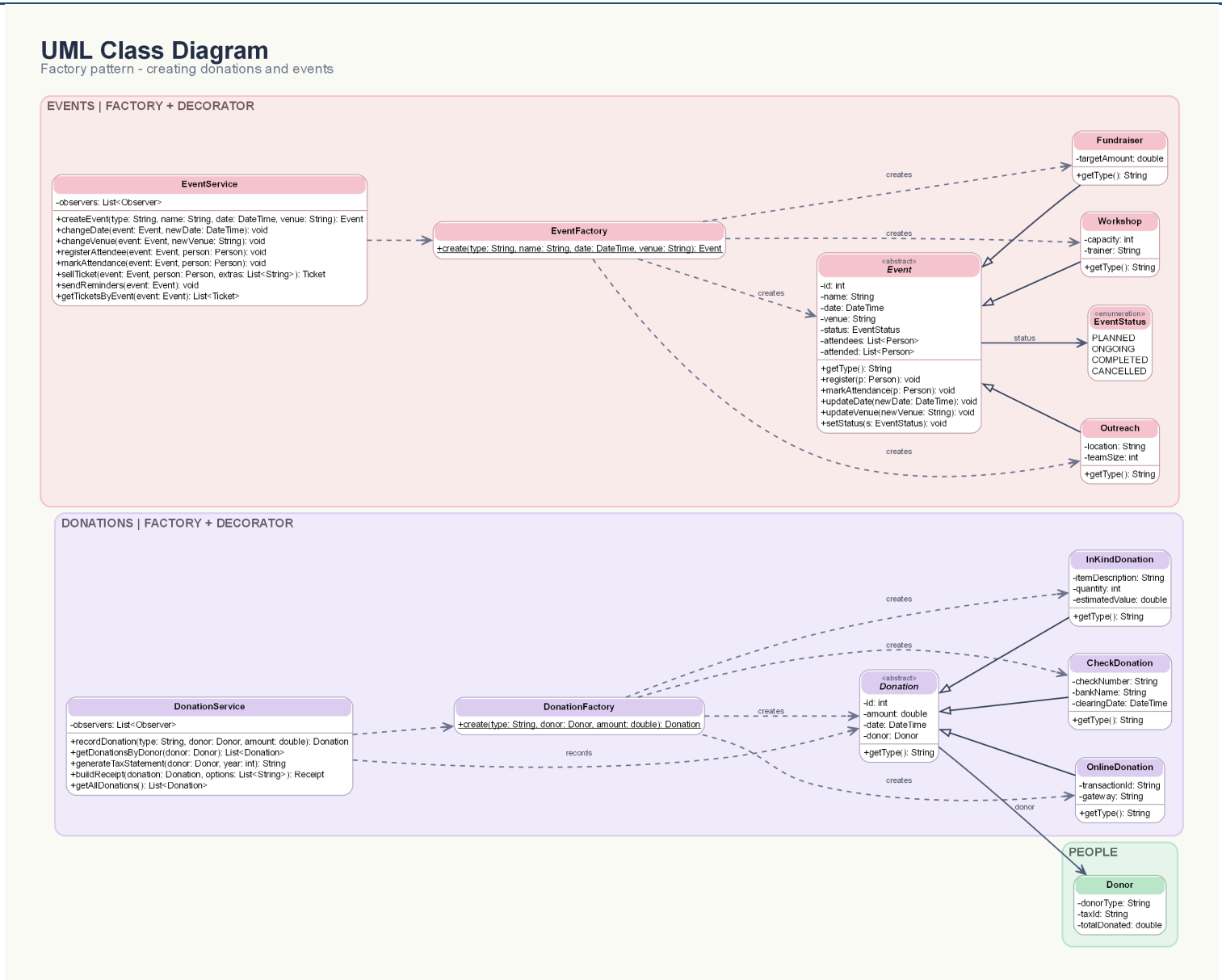


Figure 5: Factory pattern - creating donations and events

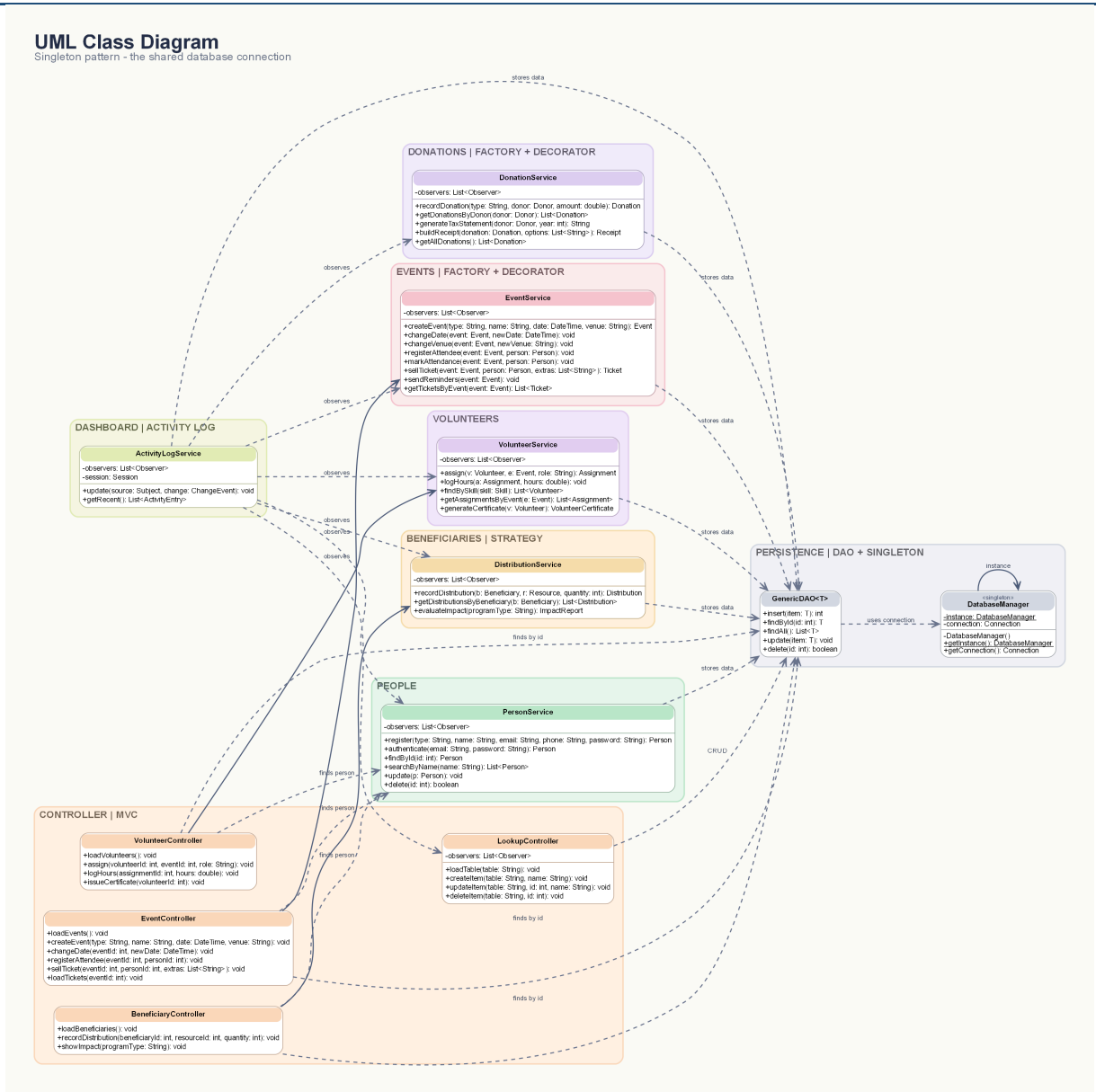


Figure 6: Singleton pattern - the shared database connection

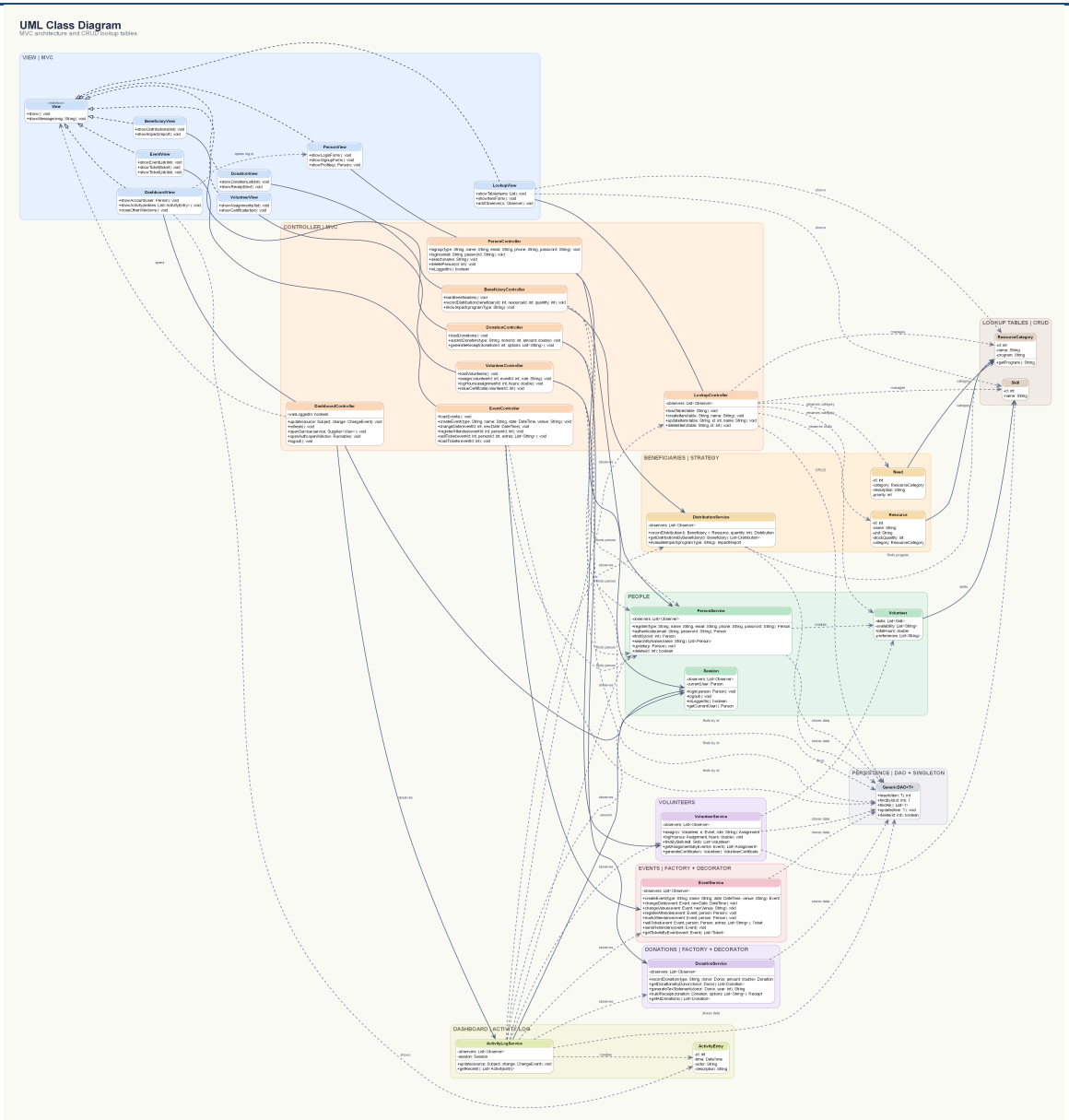


Figure 7: MVC architecture and CRUD lookup tables

6.3 Design Patterns Applied

6.3.1 Strategy (Figure 2)

Problem. Some behaviours have several interchangeable variants: how a message is delivered and how a program's impact is measured. An if/else on the variant would have to be edited every time a variant is added, breaking the open/closed principle.

Solution. Each variant is a class behind a common interface, and a context object holds a reference to one strategy and delegates to it. The pattern is applied twice:

Use	Strategy interface	Concrete strategies	Context (how the strategy is chosen)
Communication channel	NotificationStrategy	EmailStrategy, SmsStrategy, SocialMediaStrategy	NotificationService - registerStrategy() maps each Channel to a strategy; the recipient's preferredChannel picks it, and setStrategy() sets the default (e-mail)
Impact evaluation	ImpactEvaluationStrategy	FoodAidImpactStrategy, EducationImpactStrategy	ImpactEvaluator - setStrategy(); DistributionService picks the strategy for the program type chosen in BeneficiaryController, and each ResourceCategory remembers its program

```
// NotificationService (Context)
public boolean sendNotification(Person to, Message msg) {
    NotificationStrategy chosen = strategy;            // default strategy
    Channel preferred = to.getPreferredChannel();
    if (preferred != null && strategies.containsKey(preferred)) {
        chosen = strategies.get(preferred);            // recipient's channel
    }
    return chosen != null && chosen.send(to, msg);
}
```

Benefit. A WhatsApp channel is one new class, one Channel value and one registration line; a health-program impact measure is one new class and one branch where DistributionService picks the strategy. The contexts NotificationService and ImpactEvaluator do not change.

6.3.2 Observer (Figure 3)

Problem. When a donation is recorded, an event changes or resources are distributed, several independent reactions must follow (receipt, thank-you, totals, attendee messages, stock, and the dashboard's list of actions). If the services called each reaction directly they would be tightly coupled to all of them.

Solution. DonationService, EventService, DistributionService, PersonService, VolunteerService, LookupController, Session and ActivityLogService implement Subject and keep a list of Observers. Every state change creates a `ChangeEvent` (type, payload, timestamp, description) and calls `notifyObservers(change)`, which calls `update(source, change)` on each observer. `App` subscribes the observers when the application starts; ActivityLogService observes every other subject and saves each action, and DashboardController observes ActivityLogService to refresh the dashboard.

Role	Class	In the code
Subject	Subject (interface)	addObserver(), removeObserver(), notifyObservers(change)
Observer	Observer (interface)	update(source, change)

Role	Class	In the code
Concrete subjects	DonationService, EventService, DistributionService	recordDonation(); createEvent(), changeDate(), changeVenue(), registerAttendee(), markAttendance(), sellTicket(), sendReminders(); recordDistribution()
Concrete subjects	PersonService, VolunteerService, LookupController, Session, ActivityLogService	register(), delete(); assign(), logHours(), generateCertificate(); createItem(), updateItem(), deleteItem(); login(), logout(); update()
Concrete observers	ReceiptObserver, ThankYouObserver, StatsObserver	Print the receipt; thank the donor; update the donor's and the organization's totals
Concrete observers	AttendeeNotificationObserver, StockObserver	Message every attendee; reduce the resource's stock
Concrete observers	ActivityLogService, DashboardController	Save each action; refresh the dashboard

```
// DonationService (Concrete Subject)
public Donation recordDonation(String type, Donor donor, double amount) {
    Validation.requirePositive(amount, MAX_AMOUNT, "The amount");
    Donation donation = DonationFactory.create(type, donor, amount);
    donations.insert(donation);
    notifyObservers(new ChangeEvent("DONATION_RECORDED", donation, "Recorded " + donation.getType() + "
donation #"
            + donation.getId() + " of " + amount + " EGP from " + donor.getName()));
    people.update(donation.getDonor()); // save the new total set by StatsObserver
    return donation;
}
```

Benefit. Subjects only know the Observer interface, so a new reaction is one new observer; the activity log behind the dashboard was added this way, by giving each change a description and subscribing one new observer.

6.3.3 Decorator (Figure 4)

Problem. Receipts and tickets need optional extras in any combination. A subclass for every combination would cause a class explosion.

Solution. Each family has a component interface, a plain concrete component and an abstract decorator that is a component and wraps one (`wrapped`). Concrete decorators return the wrapped result plus their own part.

Family	Component	Concrete component	Decorator	Concrete decorators
Receipts	Receipt (render())	BasicReceipt	ReceiptDecorator	TaxInfoDecorator, ThankYouNoteDecorator
Tickets	Ticket (getId(), getEvent(), getPrice(), getDescription())	BasicTicket (100 EGP)	TicketDecorator	VipDecorator (+150 EGP), MealDecorator (+40 EGP)

```
// DonationService.buildReceipt()
Receipt receipt = new BasicReceipt(donation);
if (options.contains("TAX")) {
    receipt = new TaxInfoDecorator(receipt);
}
if (options.contains("THANK_YOU")) {
    receipt = new ThankYouNoteDecorator(receipt);
}
return receipt;
```

Benefit. With n decorators and m components only n + m classes are needed, and new extras (for example a signature) are one new class.

6.3.4 Factory (Figure 5)

Problem. The user picks the kind of donation (online, check, in-kind) and the kind of event (fundraiser, workshop, outreach) at runtime. If the services created them with `new` they would depend on every concrete class and repeat the choice in several places.

Solution. A factory class with a static `create(type, ...)` method is the one place that turns the user's choice into the right concrete object; the services only work with the abstract product.

Role	Donations	Events
Factory	DonationFactory.create(type, donor, amount)	EventFactory.create(type, name, date, venue)
Client	DonationService.recordDonation()	EventService.createEvent()
Product	Donation (abstract)	Event (abstract)
Concrete products	OnlineDonation, CheckDonation, InKindDonation	Fundraiser, Workshop, Outreach

```
public class DonationFactory {
    public static Donation create(String type, Donor donor, double amount) {
        return switch (type) {
            case "Online" -> new OnlineDonation(donor, amount);
            case "Check" -> new CheckDonation(donor, amount);
            case "InKind" -> new InKindDonation(donor, amount);
            default -> throw new IllegalArgumentException("Unknown donation type: " + type);
        };
    }
}
```

Benefit. The services never name a concrete donation or event class; a new type is one new product class and one line in its factory.

6.3.5 Singleton (Figure 6)

Problem. Every service and controller must work on the same data. Several database connections would waste resources and could see different data.

Solution. DatabaseManager has a private constructor, a static instance and a static synchronized `getInstance()` that creates the instance on first use (lazy initialization) and returns the same object afterwards. It holds the one JDBC connection, which every `GenericDAO<T>` uses.

```
private static DatabaseManager instance;
private final Connection connection;

private DatabaseManager() { ... connection = DriverManager.getConnection(url); }

public static synchronized DatabaseManager getInstance() {
    if (instance == null) {
        instance = new DatabaseManager();
    }
    return instance;
}
```

Benefit. One global access point for data; the database file could be replaced by a database server in Phase 2 by changing only this class.

6.4 CRUD Lookup Tables

Reference values that change over time are not hard-coded. They are rows of lookup tables that the administrator maintains from the "Lookup tables" window.

Lookup table	Row class	Used by	Sample rows
Volunteer skills	Skill	Volunteer.skills, VolunteerService.findBySkill()	First aid, Teaching, Cooking
Resource categories	ResourceCategory	Resource.category, Need.category, impact evaluation	Food, Education

Both are stored with the generic `GenericDAO<T>`, which provides `insert()`, `findById()`, `findAll()`, `update()` and `delete()` for any class with an id. `LookupController` offers `loadTable()`, `createItem()`, `updateItem()` and `deleteItem()` for either table, and `LookupView` shows the rows. Names must be unique in their table, a rename reaches the records using the row, and a row that is still in use cannot be deleted. A category created as Food or Education stays in that impact program when it is renamed.

6.5 Main Design Decisions

Option considered	Chosen design	Reason
Factory Method (one creator class per product)	Simple factories DonationFactory and EventFactory with create(type)	The brief asks for objects "of different types based on user input"; one small factory per family meets it with the fewest classes.
A separate DAO class for every table	One GenericDAO<T> that stores each object in a table named after its class	All tables get the same five CRUD operations with no extra classes; adding a new kind of record needs no new data-access code.
Data kept only in memory	SQLite file behind the DatabaseManager Singleton	Data survives restarts and the diagram's JDBC `Connection` is real; SQLite needs no server.
Console menus	Basic JavaFX windows, one per module	Each module is shown and tested on its own window with plain forms; the look is deliberately simple.
Categories stored as plain Strings	Lookup tables Skill and ResourceCategory	Editable at runtime (CRUD) without changing code.
Real e-mail and SMS providers	Channel strategies that print the message	Phase 1 needs no external accounts; real providers can replace the strategies in Phase 2 without changing NotificationService.
Each window checking the login itself	One Session object; DashboardController opens services only while someone is logged in and closes the other windows when that changes	The rule lives in one place, and a window opened earlier cannot stay usable after logging out.
Each service writing its own log entries	ActivityLogService observes every subject and saves what they announce	The services stay unaware of the dashboard; it reuses the Observer pattern already in the design.
Saving only ids inside saved records	Saved records keep a copy of what they link to; current records are updated when a lookup row is renamed or a person is deleted, and history keeps its copy	GenericDAO stays simple and old donations, receipts and tickets stay accurate; comparisons use ids, which never change.
Finding an impact program by category name	ResourceCategory remembers its program	Renaming a category does not disconnect it from its impact measure.

7. Work Distribution

Each member owns a vertical slice of the system and the pattern used in it. Every one of the 78 classes is assigned to exactly one member; the history of the GitHub repository shows each member's commits.

Member	ID	Area	Patterns	Classes
Saifeldin Tamer ElSayes `@saifeldinsais`	22P0233	Foundation, people and lookup tables: persistence, people, lookup	Singleton, CRUD lookup tables, MVC	DatabaseManager, GenericDAO, View, Person, Donor, Volunteer, Beneficiary, Skill, PersonService, PersonView, PersonController, ResourceCategory, LookupView, LookupController (and the App entry point)
Ahmed Khaled Hamdy `@ahmed-khaled04`	22P0024	Observer and notification core: observer, notification	Observer, Strategy (context)	Subject, Observer, ChangeEvent, ReceiptObserver, ThankYouObserver, StatsObserver, AttendeeNotificationObserver, StockObserver, NotificationService, NotificationStrategy, Channel, Message
Mohamed Emad Eldeen `@Mohamed-Dahy`	22P0065	Donations, notification channels and the dashboard: donation, notification.channels, dashboard	Factory, Strategy, Observer (subject and observer), MVC	DonationService, DonationFactory, Donation, OnlineDonation, CheckDonation, InKindDonation, DonationView, DonationController, EmailStrategy, SmsStrategy, SocialMediaStrategy, DashboardView, DashboardController, ActivityLogService, ActivityEntry, Session, Validation
Seif Eldeen Ibrahim `@seif-elden`	22P0023	Events and volunteer logic: event, volunteer	Factory, Observer (subject)	EventService, EventFactory, Event, Fundraiser, Workshop, Outreach, EventStatus, EventView, EventController, VolunteerService, Assignment, VolunteerController
Yousef Sameh Fathi `@YoussifSameh2004`	22P0010	Decorators and volunteer output: donation.receipt, event.ticket, volunteer	Decorator	Receipt, BasicReceipt, ReceiptDecorator, TaxInfoDecorator, ThankYouNoteDecorator, Ticket, BasicTicket, TicketDecorator, VipDecorator, MealDecorator, VolunteerView, VolunteerCertificate
Mohamed Khaled Mohamed Anwar `@tob-git`	22P0041	Beneficiaries and impact evaluation: beneficiary	Strategy, Observer (subject)	DistributionService, Distribution, Resource, Need, ImpactEvaluator, ImpactEvaluationStrategy, FoodAidImpactStrategy, EducationImpactStrategy, ImpactReport, BeneficiaryView, BeneficiaryController

8. Traceability to the Phase 1 Grading Criteria

Grading item	Marks	Diagram	Classes	Source location
Class diagram	1	Figure 1 (and all figures)	All 78 classes, interfaces and enumerations	docs/class-diagram/
Strategy pattern	0.5	Figure 2	NotificationStrategy, ImpactEvaluationStrategy and their concrete strategies; NotificationService, ImpactEvaluator	notification, notification/channels, beneficiary

Grading item	Marks	Diagram	Classes	Source location
Observer pattern	0.5	Figure 3	Subject, Observer, ChangeEvent; 8 concrete subjects; 7 concrete observers	observer, dashboard and the concrete subjects
Decorator pattern	0.5	Figure 4	Receipt + ReceiptDecorator family; Ticket + TicketDecorator family	donation/receipt, event/ticket
Factory pattern	0.5	Figure 5	DonationFactory and EventFactory with their products	donation, event
Singleton pattern	0.5	Figure 6	DatabaseManager (with GenericDAO)	persistence
MVC	0.5	Figure 7	View and 7 views, 7 controllers, App	*View, *Controller in each module; App
Client requirements	0.5	-	Section 4 (CR-01 to CR-16)	-
CRUD lookup tables	0.5	Figure 7	Skill, ResourceCategory, GenericDAO, LookupController, LookupView	lookup, people, persistence

Source paths are relative to `src/main/java/impacthub/`.

9. Running the System

9.1 Build and Run

From the project folder (requires JDK 21 and Maven, with `JAVA_HOME` pointing to JDK 21):

```
mvn javafx:run      # start the application
mvn test            # run the unit tests (they use target/test.db, never impacthub.db)
```

Alternatively, open the folder as a Maven project in IntelliJ IDEA and run the `javafx:run` goal from the Maven tool window (Plugins, javafx); running `impacthub.App` directly fails with "JavaFX runtime components are missing". Notifications and receipts are printed in the terminal (or the IDE's run console). Delete `impacthub.db` to start again with fresh sample data, or if a file from an older version can no longer be read.

9.2 Sample Data

The database is filled with sample data on the first run (every sample account's password is `1234`). Each kind of record has its own table, so ids start at 1 in each table. Nobody is logged in when the application starts:

Ids	Records
Skills 1 - 3	First aid, Teaching, Cooking
Resource categories 1 - 2	Food, Education
Resources 1 - 2	Rice bag (Food, 200 bags), School kit (Education, 100 kits)
People 1 - 2	Donors: Sara Ali (individual, e-mail, sara@mail.com), Omar Co. (corporate, SMS, omar@mail.com)
People 3 - 4	Volunteers: Ali Hassan (First aid; weekends; prefers outreach; e-mail), Mona Adel (Teaching; evenings; prefers workshops; social media)
People 5 - 6	Beneficiaries: Fatma Said (Cairo, family of 5, needs food), Hassan Omar (Giza, family of 3, needs school supplies)

Ids	Records
Events 1 - 2	Charity Gala (fundraiser, in 2 weeks, Grand Hall), First Aid Training (workshop, in 1 week, Community Center)

9.3 Demonstration Scenario

1. **Dashboard:** start the application; it opens on the dashboard with the service buttons disabled. Log in as sara@mail.com / 1234 and the buttons unlock. Log out, then sign up a new volunteer, which logs them in. Log out and log in as Sara again.
2. **People:** search with an empty name to see everyone's ids; try to delete person 1, yourself (refused); delete the new person.
3. **Donations:** record an Online donation of 500 EGP for donor 1. The terminal shows the thank-you e-mail and the receipt (Observer), and the window shows the donor's new total. Record a Check donation for donor 2 - the thank-you arrives by SMS (Strategy). Then generate a receipt for donation 1 with tax information and a thank-you note (Decorator).
4. **Events:** create a Workshop (Factory); register person 1 for event 1; sell person 2 a ticket for event 1 with VIP seating and dinner (290 EGP) and list the event's tickets; change event 1's date - every attendee is notified in the terminal (Observer).
5. **Volunteers:** list volunteers; assign volunteer 3 to event 2 as "First aider"; log 4 hours on that assignment; issue a certificate for volunteer 3.
6. **Beneficiaries:** give 10 rice bags (resource 1) to beneficiary 5 and see the stock drop to 190 (Observer); try 500 bags (refused); give 1 school kit (resource 2) to beneficiary 6; evaluate the Food and the Education impact (Strategy).
7. **Lookup tables:** load the Skill table, create "Driving", rename it and delete it; do the same for a resource category; try to delete skill 1, which is in use (refused).
8. **Dashboard:** see the last 10 actions, newest first (Observer). Log out and every open window closes. Close the dashboard to exit, start again and log in: the actions are still there.